

Introduction to SQL

Miao Qiao

The University of Auckland



Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<u><i>s_id</i></u> <u><i>i_id</i></u>

Basic Query Structure (demo)

1. Find the names of all instructors
2. Find the department names of all instructors, and remove duplicates
3. Find the department names of all instructors, not removing duplicates
4. Find all attributes of instructor show the entire instructor table
5. Find a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12
6. Find all instructors in Comp. Sci. dept with salary > 70000
7. Find the names of all instructors who have taught some course and the course_id
8. Find the names of all instructors in the Comp. Sci. department who have taught some course and the course_id
9. Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

				section				
department	instructor	teaches			takes	student	course	advisor
<u>dept_name</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>course_id</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	
building	name	<u>sec_id</u>	<u>sec_id</u>	<u>sec_id</u>	<u>course_id</u>	name	title	<u>s_id</u>
	dept_name	<u>semester</u>	<u>semester</u>	<u>semester</u>	<u>sec_id</u>	dept_name	dept_name	<u>i_id</u>
	salary	<u>year</u>	<u>year</u>	<u>year</u>	<u>semester</u>	tot_cred	credits	
				<u>building</u>	<u>year</u>			
				<u>room_number</u>	<u>grade</u>			
				<u>time_slot_id</u>				

Basic Query Structure (demo)

- Find the names of all instructors whose name includes the substring “in”.
- String Operations
 - The operator **like** uses patterns that are described using two special characters:
 - percent (%). The % character matches any substring.
 - underscore (_). The _ character matches any character
- Find the names of all instructors whose name has 4 characters.
- Find the names of all instructors whose name has at least 4 characters.

				section				
department	instructor	teaches			takes	student	course	advisor
<u>dept_name</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>course_id</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	
building	name	<u>sec_id</u>	<u>sec_id</u>	<u>sec_id</u>	<u>course_id</u>	name	title	<u>s_id</u>
	dept_name	<u>semester</u>	<u>semester</u>	<u>semester</u>	<u>sec_id</u>	dept_name	dept_name	<u>i_id</u>
	salary	<u>year</u>	<u>year</u>	<u>year</u>	<u>semester</u>	tot_cred	credits	
				building	<u>year</u>			
				room_number	grade			
				time_slot_id				

Basic Query Structure (demo)

1. List in alphabetic order the names of all instructors
2. List in descending alphabetic order the names of all instructors
3. List in order of the combination of the names and salary of all instructors
4. Find the names of all instructors with salary between \$90,000 and \$100,000
5. Find courses that ran in Fall 2017 or in Spring 2018
6. Find courses that ran in Fall 2017 and in Spring 2018
7. Find courses that ran in Fall 2017 but not in Spring 2018
8. Find courses that ran in Fall 2017 or in Spring 2018, retain all duplications
9. Find all instructors whose salary is null
10. Find all instructors whose salary is not null
11. Null under and, or, with true/false ?

*select null and true
false.
or xx
or xx*

department		instructor		teaches	section	takes	student	course	advisor
<u>dept_name</u>		<u>ID</u>	<u>name</u>	<u>ID</u>	<u>course_id</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>s_id</u>
building		dept_name		course_id	sec_id	course_id	name	title	i_id
budget		salary		sec_id	semester	sec_id	dept_name	dept_name	
				semester	year	semester	tot_cred	credits	
				year	building	year			
					room_number	grade			
					time_slot_id				

Basic Query Structure (demo)

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

select $A_1, A_2, \dots, A_n$  Aggregation function over values over multiple rows

from $r_1, r_2, \dots, r_m$

where P

 every single row

group by columns  New clauses

having condition

 every group

department		instructor		teaches		section	takes		student	course		advisor
<u>dept_name</u>		<u>ID</u>		<u>ID</u>		<u>course_id</u>	<u>ID</u>		<u>ID</u>	<u>course_id</u>		<u>s_id</u>
building		name		course_id		sec_id	course_id		name	title		i_id
		dept_name		sec_id		semester	sec_id		dept_name	dept_name		
		salary		semester		building	semester		tot_cred	credits		
				year		room_number	year					
				year		time_slot_id	grade					

Basic Query Structure (demo)

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

- Find the highest salary of any instructor. *select max(salary) from ...*
- Find the average salary of instructors in the Computer Science department *select avg(salary) from xx where xx*
- Find the lowest salary of an instructor who have taught a course
- Find the total number of instructors who teach a course in the Spring 2018 semester *select count(distinct ID) from teaches where semester = 'Spring' and year = 2018*
- Find the number of tuples in the *course* relation

department		instructor		teaches	section	takes	student	course	advisor
<u>dept_name</u>		<u>ID</u>	name	<u>ID</u>	<u>course_id</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>s_id</u>
building		dept_name		course_id	sec_id	course_id	name	title	i_id
budget		salary		semester	year	sec_id	dept_name	dept_name	
				year	building	semester	tot_cred	credits	
					room_number	year			
					time_slot_id	grade			

Aggregate – Group By - Having

1. Find the average salary of instructors in each department
select dept_name, avg(salary) from instructor group by dept_name
2. Find the names and average salaries of all departments whose average salary is greater than 42000
having avg(salary) > 42000 from instructor where salary > 7000 group by ...
3. Find the names and average salaries of all departments over instructors whose salary is greater than 7000
4. Find the names and average salaries of all departments whose average salary is greater than 70000
5. Find the average salaries of instructors who have taught a course

*select avg(salary) from instructors where id
in (select id from instructor natural join teaches)*

department		instructor		teaches		section		takes		student		course		advisor	
<u>dept_name</u>		<u>ID</u>	name	<u>ID</u>	<u>course_id</u>	<u>sec_id</u>	<u>semester</u>	<u>ID</u>	<u>course_id</u>	<u>ID</u>	name	<u>course_id</u>	title	<u>s_id</u>	<u>i_id</u>
building		dept_name		<u>sec_id</u>	<u>semester</u>	building	room_number	<u>sec_id</u>	<u>semester</u>	dept_name		dept_name	credits		
budget		salary		<u>year</u>	<u>year</u>	time_slot_id		<u>year</u>	grade	tot_cred					

Nested Subqueries

- A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select A1, A2, ..., An
from r1, r2, ..., rm
where P
```

" in "

as follows:

- From clause:** r_i can be replaced by any valid subquery
- Where clause:** P can be replaced with an expression of the form:
 $B <\text{operation}> (\text{subquery})$

B is an attribute and $<\text{operation}>$ to be defined later.

- Select clause:**

A_i can be replaced by a subquery that generates a single value.

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Subquery in Where Clause

*select * from instructor where name in ('Mozart', 'Einstein');*

1. Name all instructors whose name is either "Mozart" or Einstein"
2. Name all instructors whose name is neither "Mozart" nor Einstein"
3. Find the total number of (distinct) students who have taken course sections taught by the instructor with ID 10101 or 12121 or 15151

4. Find names of instructors with salary greater than that of some (at least one) instructor in the Computer Science department.
select name from instructor where salary > some (select salary from instructor where dept_name = 'cs')

5. Find all instructors earning the highest salary (there may be more than one with the same salary).

4的some改成all, >改成>=

6. Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

5的subqua + where ...

3. *D select count(distinct B.id) from teaches A, takes B where (A.course_id, A.sec_id, A.semester, A.year) = (B.course_id, B.sec_id, B.semester, B.year) and A.id in ('10101', '12121', '15151');*

count(DISTINCT B.id)
int64

department		instructor		teaches		section		takes		student		course		advisor	
<u>dept_name</u>		<u>ID</u>		<u>ID</u>		<u>course_id</u>		<u>ID</u>		<u>ID</u>		<u>course_id</u>		<u>s_id</u>	
building		name		course_id		sec_id		course_id		name		title		i_id	
		dept_name		sec_id		semester		sec_id		dept_name		dept_name			
budget		salary		semester		building		semester		tot_cred		credits			
				year		room_number		year							
						time_slot_id		grade							

Definition of “some” Clause

- $F <\text{comp}> \text{some } r \Leftrightarrow \exists t \in r \text{ such that } (F <\text{comp}> t)$

Where $<\text{comp}>$ can be: $<$, $\leq$, $>$, $=$, $\neq$

$$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true} \quad (\text{read: } 5 < \text{some tuple in the relation})$$

$$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

其中-个

$$(5 = \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$$

$$(5 \neq \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true (since } 0 \neq 5)$$

$(= \text{some}) \equiv \text{in}$

However, $(\neq \text{some}) \not\equiv \text{not in}$

Definition of “all” Clause

- $F \text{ <comp> all } r \Leftrightarrow \forall t \in r (F \text{ <comp> } t)$

$$(5 < \text{all} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \text{all} \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all} \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all} \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$$

$(\neq \text{all}) \equiv \text{not in}$

However, $(= \text{all}) \not\equiv \text{in}$

distinct course_id.

Subquery in Where Clause

empty

false

→ text weather empty

not empty

true

- | | | | | | | | |
|-------------------------|-------------------|------------------------|-------------------------|-------------------------|------------------|-------------------------|--------------------|
| <i>department</i> | <i>instructor</i> | <i>teaches</i> | <i>section</i> | <i>takes</i> | <i>student</i> | <i>course</i> | <i>advisor</i> |
| <u><i>dept_name</i></u> | <u><i>ID</i></u> | <u><i>ID</i></u> | <u><i>course_id</i></u> | <u><i>ID</i></u> | <u><i>ID</i></u> | <u><i>course_id</i></u> | <u><i>s_id</i></u> |
| <i>building</i> | <i>name</i> | <u><i>sec_id</i></u> | <u><i>semester</i></u> | <u><i>course_id</i></u> | <i>name</i> | <i>title</i> | <u><i>i_id</i></u> |
| <i>budget</i> | <i>dept_name</i> | <u><i>semester</i></u> | <i>building</i> | <u><i>sec_id</i></u> | <i>dept_name</i> | <i>dept_name</i> | |
| | <i>salary</i> | <u><i>year</i></u> | <i>room_number</i> | <u><i>semester</i></u> | <i>tot_cred</i> | <i>credits</i> | |
| | | | <i>time_slot_id</i> | <u><i>year</i></u> | | | |
| | | | | <i>grade</i> | | | |

distinct
select name from instructor A and instructor B where
Query Quest

A. salary > B. salary and B. dept_name = 'Comp. Sci.'

```
select B.name from instructor B where exists (select * from instructor A where A.dept_name = 'Comp. Sci.' and B.salary > A.salary);
```

1. Find names of instructors with salary greater than that of some (at least one) instructor in the Computer Science department.

- Use self-join, some, exists, aggregation 集合

2. Find all instructors earning the highest salary (there may be more than one with the same salary). select * from instructor where salary

- Use self-join, all, exists, aggregation >= all (select salary from

select name from instructor where salary > (select instructor, min(salary) from instructor where dept_name = 'Comp. Sci.') order by name

department	instructor	teaches	section	takes	student	course	advisor
dept_name	ID name dept_name salary	ID course_id sec_id semester year	course_id sec_id semester year building room_number time_slot_id	ID course_id sec_id semester year grade	ID name dept_name tot_cred	course_id title dept_name credits	s_id i_id

Subquery in From and Select Clauses

select avg_salary from (select dept_name, avg(salary) as

*avg_salary from instructor
group by dept_name)*

where avg_sal

> 42000

(select max(budget from department)

where budget = max-budget.value;

1. Find the average instructors' salaries of those departments where the average salary is greater than \$42,000
2. Find all departments with the maximum budget (with clause)
3. Find all departments where the total salary is greater than the average of the total salary at all departments
4. List all departments along with the number of instructors in each department

```
with totsalarly(dept_name, value) as (select dept_name, sum(salary) from instructor group by dept_name), avgsalary(avgvalue) as (select avg(value) from totsalarly) select * from totsalarly, avgsalary where value > avgsalary.avgvalue;
```

dept_name varchar	value double	avgvalue double
Comp. Sci.	232000.0	128285.71428571429
Physics	182000.0	128285.71428571429
Finance	170000.0	128285.71428571429

section

4.

```
select B.dept_name, (select count(*) from instructor A where A.dept_name = B.dept_name) from department B;
```

dept_name varchar	(SELECT count_star() FROM instructor AS A WHERE (A.dept_name = B.dept_name)) int64
Biology	1
Comp. Sci.	3
Elec. Eng.	1
Finance	2
History	2
Music	1
Physics	2

student

course

ID
name
dept_name
tot_cred

course_id
title
dept_name
credits

advisor

s_id
i_id

With Clause

- The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.
- Find all departments with the maximum budget

```
with max_budget (value) as
    (select max(budget)
     from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Query Quest

- Find all departments where the total salary is greater than the average of the total salary at all departments

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Modification of the Database

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

tuple

- Delete all instructors

delete from *instructor*

- Delete all instructors from the Finance department

delete from *instructor*

where *dept_name* = 'Finance';

- Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.

delete from *instructor*

where *dept name* in (**select** *dept name*
from *department*
where *building* = 'Watson');

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<u><i>s_id</i></u> <u><i>i_id</i></u>

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<i>s_id</i> <i>i_id</i>

Insertion

- Add a new tuple to *course*

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to *student* with *tot_creds* set to null

insert into *student*

values ('3003', 'Green', 'Finance', *null*);

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <i>year</i> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<u><i>s_id</i></u> <u><i>i_id</i></u>

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

批量插入

```
insert into instructor
select ID, name, dept_name, 18000
from student
where dept_name = 'Music' and total_cred > 144;
```

- The **select from where** statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

insert into table1 select * from table1
would cause problem

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u>	<u><i>ID</i></u>	<u><i>ID</i></u>	<u><i>course_id</i></u>	<u><i>ID</i></u>	<u><i>ID</i></u>	<u><i>course_id</i></u>	<u><i>s_id</i></u>
<i>building</i>	<i>name</i>	<u><i>sec_id</i></u>	<u><i>sec_id</i></u>	<u><i>course_id</i></u>	<i>name</i>	<i>title</i>	<u><i>i_id</i></u>
<i>budget</i>	<i>dept_name</i>	<u><i>semester</i></u>	<i>year</i>	<u><i>sec_id</i></u>	<i>dept_name</i>	<i>dept_name</i>	
	<i>salary</i>	<u><i>year</i></u>	<i>building</i>	<u><i>semester</i></u>	<i>tot_cred</i>	<i>credits</i>	
			<i>room_number</i>	<u><i>year</i></u>			
			<i>time_slot_id</i>	<i>grade</i>			

- Give a 5% salary raise to all instructors

```
update instructor  
  set salary = salary * 1.05
```

- Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor
  set salary = salary * 1.05
where salary < 70000;
```

- Give a 5% salary raise to instructors whose salary is less than average

```
update instructor
set salary = salary * 1.05
where salary < (select avg (salary)
                  from instructor);
```

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<i>s_id</i> <i>i_id</i>

Updates (Cont.)

- Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%

- Write two **update** statements:

```
update instructor
  set salary = salary * 1.03
  where salary > 100000;
update instructor
  set salary = salary * 1.05
  where salary <= 100000;
```

order! 不能反。
否则有些老师的涨2次

- The order is important
- Can be done better using the **case** statement (next slide)

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Case Statement for Conditional Updates

- Same query as before but with case statement

update *instructor*

set *salary* = **case**

when *salary* <= 100000 **then** *salary* * 1.05

else *salary* * 1.03

end

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <i>year</i> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<i>s_id</i> <i>i_id</i>

value.

Updates with Scalar Subqueries

- Recompute and update tot_creds value for all students

update *student* *S*

set *tot_cred* = (**select** **sum**(*credits*)

from *takes*, *course*

where *takes.course_id* = *course.course_id* **and**

S.ID = *takes.ID* **and**

takes.grade <> 'F' **and**

takes.grade **is not null**);

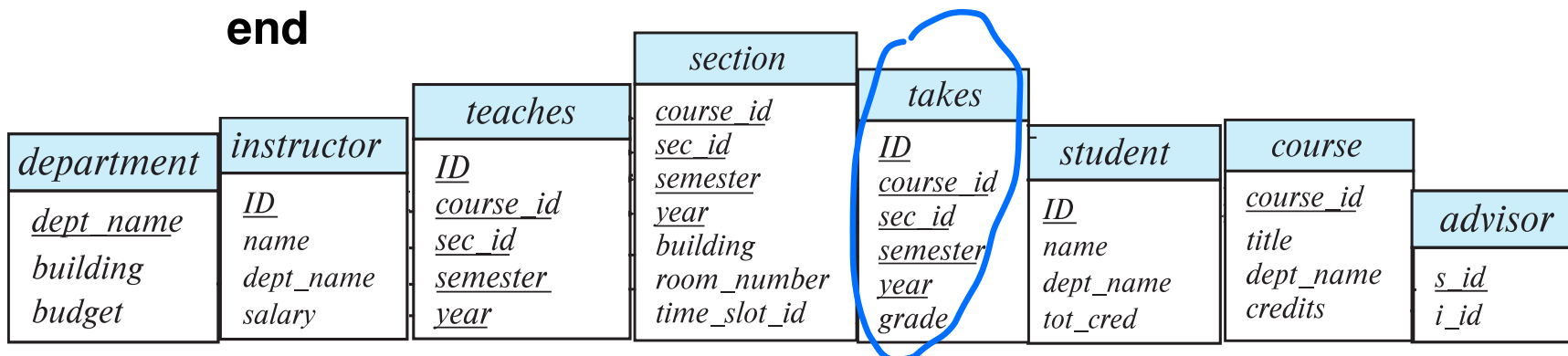
- Sets *tot_creds* to null for students who have not taken any course
- Instead of **sum**(*credits*), use:

case

when **sum**(*credits*) **is not null** **then** **sum**(*credits*)

else 0

end



FIN

Any questions?

Join types

- Three forms of outer join:
 - Natural left outer join *course* ⋈ *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

<i>department</i>		<i>teaches</i>		<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<i>dept_name</i>	<i>ID</i>	<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>ID</i>	<i>ID</i>	<i>course_id</i>	
<i>building</i>	<i>name</i>	<i>sec_id</i>	<i>sec_id</i>	<i>semester</i>	<i>course_id</i>	<i>name</i>	<i>title</i>	<i>s_id</i>
<i>budget</i>	<i>dept_name</i>	<i>semester</i>	<i>semester</i>	<i>building</i>	<i>sec_id</i>	<i>dept_name</i>	<i>dept_name</i>	<i>i_id</i>
	<i>salary</i>	<i>year</i>	<i>year</i>	<i>room_number</i>	<i>semester</i>	<i>tot_cred</i>	<i>credits</i>	
				<i>time_slot_id</i>	<i>year</i>			
					<i>grade</i>			

Join types

- Three forms of outer join:
 - Natural left outer join *course* ⋈ *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <i>year</i> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<i>s_id</i> <i>i_id</i>

Join types

Three forms of outer join:

- Natural left outer join *course* $\bowtie$ *prereq*
- Natural right outer join *course* $\bowtie$ *prereq*
- Natural full outer join *course* $\bowtie$ *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<i>s_id</i> <i>i_id</i>

Joined Relations – Examples

- course natural right outer join prereq

course_id	title	dept_name	credits	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	null	null	null	CS-101

- course full outer join prereq using (course_id)

course_id	title	dept_name	credits	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	null
CS-347	null	null	null	CS-101

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> i_id

Joined Relations – Examples

- remain all

		section					
		teaches		takes			
department	instructor		course_id		student	course	
<u>dept_name</u>	<u>ID</u>	<u>ID</u>	<u>sec_id</u>	<u>ID</u>		<u>course_id</u>	advisor
building	name	course_id	semester	course_id	<u>ID</u>	title	
	dept_name	<u>sec_id</u>	building	<u>sec_id</u>	name	dept_name	<u>s_id</u>
	salary	<u>semester</u>	room_number	<u>semester</u>	tot_cred	credits	<u>i_id</u>
		<u>year</u>	time_slot_id	<u>year</u>			
				grade			

Virtual

view 可嵌套

不真实存在

每次执行,更新从表

<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<u><i>s_id</i></u> <u><i>i_id</i></u>

Join

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on < predicate >
using ($A_1, A_2, \dots, A_n$)

<u>course_id</u>	<u>title</u>	<u>dept_name</u>	<u>credits</u>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

<u>course_id</u>	<u>prereq_id</u>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

department		instructor		teaches		section		takes		student		course		advisor									
<u>dept_name</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>course_id</u>	<u>sec_id</u>	<u>semester</u>	<u>year</u>	<u>ID</u>	<u>course_id</u>	<u>sec_id</u>	<u>semester</u>	<u>year</u>	<u>grade</u>	<u>ID</u>	<u>name</u>	<u>dept_name</u>	<u>tot_cred</u>	<u>course_id</u>	<u>title</u>	<u>dept_name</u>	<u>credits</u>	<u>s_id</u>	<u>i_id</u>
building	dept_name	name	course_id	sec_id	semester	year	building	room_number	course_id	sec_id	semester	year	grade	name	dept_name	tot_cred		course_id	title	dept_name	credits		
budget	salary		year				time_slot_id																

Joined Relations – Examples

- *course* **inner join** *prereq* **on** *course.course_id = prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

- What is the difference between the above, and a natural join?
- *course* **left outer join** *prereq* **on** *course.course_id = prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	<i>null</i>	<i>null</i>

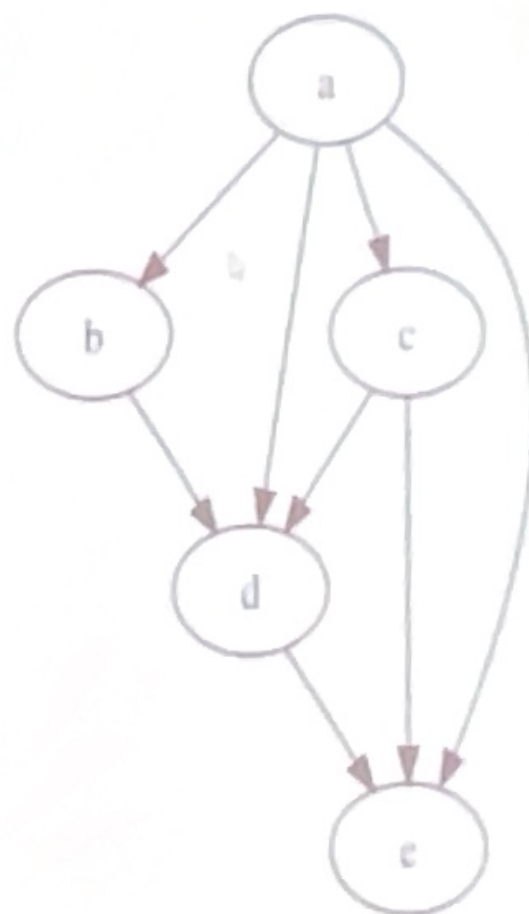
<i>department</i>	<i>instructor</i>	<i>teaches</i>	<i>section</i>	<i>takes</i>	<i>student</i>	<i>course</i>	<i>advisor</i>
<u><i>dept_name</i></u> <i>building</i> <i>budget</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>salary</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u>	<u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>building</i> <i>room_number</i> <i>time_slot_id</i>	<u><i>ID</i></u> <u><i>course_id</i></u> <u><i>sec_id</i></u> <u><i>semester</i></u> <u><i>year</i></u> <i>grade</i>	<u><i>ID</i></u> <i>name</i> <i>dept_name</i> <i>tot_cred</i>	<u><i>course_id</i></u> <i>title</i> <i>dept_name</i> <i>credits</i>	<u><i>s_id</i></u> <u><i>i_id</i></u>

Query Quest

1. Display a list of all instructors, showing their ID, name, and the number of sections that they have taught. Make sure to show the number of sections as 0 for instructors who have not taught any section. Your query should use an outer join, and should not use scalar subqueries.
2. Outer join expressions can be computed in SQL without using the SQL outer join operation. To illustrate this fact, show how to rewrite each of the following SQL queries without using the outer join expression.
 1. **select * from student natural left outer join takes**
 2. (hard) **select * from student natural full outer join takes**
 1. Select id, name, count(coalesce(course_id, sec_id, semester, year)) from instructor natural left outer join teaches group by id;

department		instructor		teaches		section	takes		student	course	advisor	
<u>dept_name</u>		<u>ID</u>	name	<u>ID</u>	<u>course_id</u>	<u>course_id</u>	<u>ID</u>	<u>course_id</u>	<u>ID</u>	<u>course_id</u>		
building		dept_name		sec_id	sec_id	sec_id	sec_id	sec_id	name	title	<u>s_id</u>	
budget		salary		semester	semester	semester	semester	semester	dept_name	dept_name	i_id	
				year	year	year	year	year	tot_cred	credits		
						building		grade				
						room_number						
						time_slot_id						

Dependency Graph of Views should be a DAG

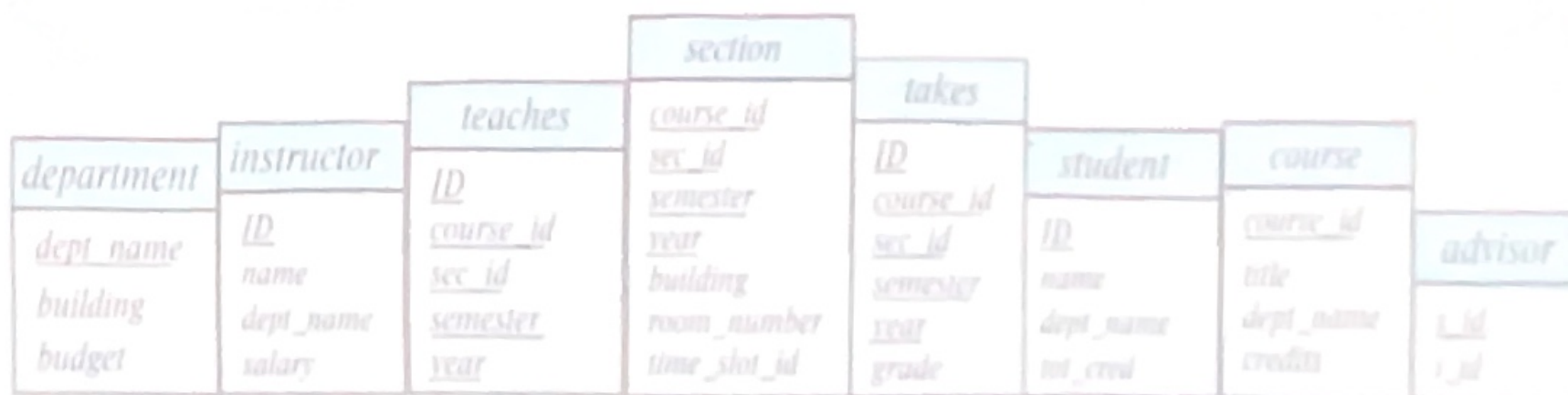


department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Integrity Constraints

完整性约束

- A checking account must have a balance greater than \$10,000.00
- A salary of a bank employee must be at least \$4.00 an hour
- A customer must have a (non-null) phone number



Constraints on a Single Relation

- not null
 - *name* varchar(20) not null
 - *budget* numeric(12,2) not null
- primary key
- unique ($A_1, A_2, \dots, A_m$)
 - Candidate keys are permitted to be null (in contrast to primary keys).
- check (P), where P is a predicate (read DDL.sql)

create table department

(dept_name varchar(20), building varchar(15),
 budget numeric(12,2) **check** (budget > 0),
primary key (dept_name));

department		instructor		teaches		section		takes		student		course		advisor	
<u>dept_name</u>		<u>ID</u>		<u>ID</u>		<u>course_id</u>		<u>ID</u>		<u>ID</u>		<u>course_id</u>		<u>i_id</u>	
building		name		course_id		sec_id		course_id		name		title		i_id	
budget		dept_name		sec_id		year		sec_id		dept_name		dept_name		i_id	
		salary		semester		building		semester		tot_cred		credit			
				year		room_number		year							
						time_slot_id		grade							

Integrity Constraint Violation

- create table *person* (
 ID char(10),
 name char(40),
 mother char(10),
 father char(10),
 primary key *ID*,
 foreign key *father* references *person*,
 foreign key *mother* references *person*)
- How to insert a tuple without causing constraint violation?

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Built-in Data Types in SQL

- **date**: Example: **date** '2005-7-27'
- **time**: Example: **time** '09:00:30' **time** '09:00:30.75'
- **Timestamp** Example: **timestamp** '2005-7-27 09:00:30.75'
- **interval**: Subtracting a date/time/timestamp value from another gives an interval value
- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*:
 - **blob**: binary large object
 - **clob/text**: character large object
 - When a query returns a large object, a pointer is returned rather than the large object itself.

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>

Index Creation

- We create an index with the **create index** command
create index <name> on <relation-name> (attribute);

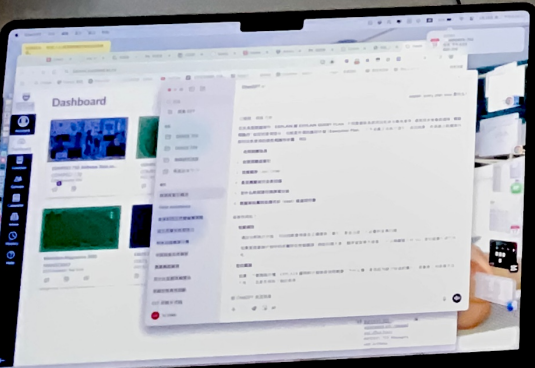
			section				
department	instructor	teaches		takes	student	course	advisor
<u>dept_name</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	<u>ID</u>	<u>ID</u>	<u>course_id</u>	
building	name	<u>course_id</u>	<u>sec_id</u>	<u>course_id</u>	name	title	<u>s_id</u>
	dept_name	<u>sec_id</u>	<u>semester</u>	<u>sec_id</u>	dept_name	dept_name	<u>i_id</u>
budget	salary	<u>semester</u>	<u>year</u>	<u>semester</u>	tot_cred	credits	
		<u>year</u>	building	<u>year</u>			
			room_number	grade			
			time_slot_id				


```

sqlite> select * from student;
+-----+-----+-----+-----+
| ID   | name  | dept_name | tot_cred |
+-----+-----+-----+-----+
| 00128 | Zhang | Comp. Sci. | 102      |
| 12345 | Shankar | Comp. Sci. | 32       |
| 19991 | Brandt | History    | 80       |
| 23121 | Chavez | Finance    | 110      |
| 44553 | Peltier |            | 56       |
| 45678 | Levy   |            | 46       |
| 54321 | Williams | Comp. Sci. | 54       |
| 55739 | Sanchez | Music      | 38       |
| 70557 | Snow   |            | 0        |
| 76543 | Brown  | Comp. Sci. | 58       |
| 76653 | Aoi    | Elec. Eng. | 60       |
| 98765 | Bourikas | Elec. Eng. | 98       |
| 98988 | Tanaka | Biology    | 120      |
+-----+-----+-----+-----+

sqlite> explain query plan select * from student where id = '54321';
QUERY PLAN
|--SEARCH student USING INDEX sqlite_autoindex_student_1 (ID=?)
sqlite>

```



Ino

- V
- c
- d
- p
- c
- T

Demo

depar
degr
build
hang

Transactions

事項、仕事

- The transaction must end with one of the following statements:
 - **Commit work.**
 - **Rollback work.**
- Atomic transaction
- Isolation from concurrent transactions
- ACID
 - Consistency and Durability.
- Demo: transaction

department	instructor	teaches	section	takes	student	course	advisor
<u>dept_name</u> building budget	<u>ID</u> name dept_name salary	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u>	<u>course_id</u> <u>sec_id</u> <u>semester</u> year building room_number time_slot_id	<u>ID</u> <u>course_id</u> <u>sec_id</u> <u>semester</u> <u>year</u> grade	<u>ID</u> name dept_name tot_cred	<u>course_id</u> title dept_name credits	<u>s_id</u> <u>i_id</u>